

Техническое задание

Система поиска аналогов товаров на маркетплейсах

1 Назначение и объём проекта

Разработать программу на Python для сбора товаров трёх маркетплейсов, хранения данных в PostgreSQL и подбора аналогов с помощью SQL-запросов. Основной результат — база данных, её схема и набор запросов для фильтрации, сравнения товаров и анализа цен.

MVP поддерживает одну категорию — наушники. Базовые площадки: Ozon, Wildberries и Яндекс Маркет; окончательный состав определяется после проверки доступности данных. Пользователь передаёт ID сохранённого товара и параметры поиска, программа возвращает подходящие товары с других площадок.

Обязательный интерфейс — командная строка и экспорт CSV. Web, REST API и расширение Chrome относятся к возможному развитию. Полный сбор каталогов не требуется.

2 Технологии и структура

Использовать Python 3.12, PostgreSQL 17 или 18 и psycorg для SQL. Зависимости закрепить в requirements.txt. Существующий Playwright использовать только при необходимости загрузки динамических страниц. Для сохранённого HTML допускается одна библиотека разбора — BeautifulSoup.

Программа выполняется последовательно в одном процессе. Очереди, Redis, Celery, ORM, машинное обучение и отдельный frontend не требуются. Схема создаётся и изменяется нумерованными SQL-файлами.

Файл или каталог	Назначение
main.py	Команды и запуск сценариев
parsers/	Получение данных площадок
normalize.py	Очистка цен, названий и характеристик
db.py	Подключение к PostgreSQL и запись
sql/schema.sql	Таблицы, ограничения и индексы
sql/queries/	Пользовательские SQL-запросы
data/ и tests/	Примеры и проверки

Подключение читать из DATABASE_URL. Секреты не сохранять в коде. Значения передавать через параметры psycorg; имена запросов и сортировки выбирать из разрешённого списка.

3 Требования к базе данных

3.1 Таблицы

Таблица	Основные поля
marketplaces	id, code UNIQUE, name
products	id, marketplace_id, external_id, name, url, brand, model, connection_type, form_factor, anc, microphone, battery_hours, first_seen_at, last_seen_at
parse_runs	id, marketplace_id, source_url, started_at, finished_at, status, error_message
price_history	id, product_id, run_id, observed_at, price, currency, region, price_kind, availability, rating, reviews_count

Одна строка products — одна карточка площадки. Уникальность: (marketplace_id, external_id). Общий каталог моделей не требуется; соответствие карточек определяется запросом. Связи marketplace_id, product_id и run_id оформить внешними ключами.

ID — BIGINT, внешний ID — TEXT, цена — NUMERIC(14,2), время — TIMESTAMPTZ, anc и microphone — BOOLEAN с допустимым NULL. В Python суммы обрабатывать через Decimal. Неизвестные признаки сохранять как NULL, а не false или 0.

connection_type: wired, bluetooth, radio; form_factor: in_ear, on_ear, over_ear. Неизвестное значение — NULL. Автономность хранить в часах. При очистке model сохранять цифры и обозначения версии. Исходное название не заменять нормализованным.

price_kind: regular, card, subscription, unknown. availability: in_stock, out_of_stock, unknown. region — код региона либо unknown. Цена допускает NULL, известная цена неотрицательна. Рейтинг — 0–5 либо NULL, число отзывов — неотрицательное либо NULL.

3.2 Запись и история

Каждый сбор создаёт parse_runs. Карточки сохраняются через INSERT ... ON CONFLICT. Повторный сбор обновляет карточку и добавляет price_history, сохраняя прошлые цены. Уникальное ограничение запрещает повтор наблюдения одного товара в одном запуске для одинаковых региона, валюты и типа цены.

Результат небольшого запуска записывается одной транзакцией после получения данных. Ошибка откатывает запись; запуск помечается failed отдельной транзакцией. Другие площадки продолжают обработку. Ошибка загрузки не заменяет известные данные пустыми значениями.

Создать представление current_prices: последнее наблюдение по товару, региону, валюте и типу цены. Сначала выбирается последнее наблюдение, затем применяются фильтры. Последние NULL price и out_of_stock не заменять старой выгодной ценой.

3.3 Индексы

Обязательны: уникальный (marketplace_id, external_id); products по (connection_type, form_factor); price_history по (product_id, region, currency, price_kind, observed_at DESC); индекс run_id. Дополнительные индексы вводить после EXPLAIN ANALYZE. pg_trgm и JSONB для MVP не обязательны.

4 Функции и подбор аналогов

Программа загружает данные, нормализует их, записывает в БД, выполняет выбранный запрос и выводит таблицу либо CSV. Предусмотреть импорт JSON для повторяемых проверок независимо от доступности сайта.

Команды: collect — сбор с выбранной или всех площадок; import — загрузка JSON; find — подбор по ID товара; query — выполнение именованного запроса с параметрами; export — сохранение результата в CSV.

Базовый подбор выполняется SQL: другая площадка, одинаковые connection_type и form_factor, выполнение обязательных условий пользователя. ANC и микрофон становятся обязательными по выбору пользователя. Неизвестный обязательный признак исключает кандидат из подходящих аналогов.

Бренд может отличаться. Дополнительные фильтры: бренд, цена, рейтинг, наличие и возраст данных. Сортировка — по цене либо количеству совпавших известных признаков, затем по ID. Сходство названий не требуется для основной логики.

Режим «Та же модель» сравнивает нормализованные бренд и модель. Если модель неизвестна, совпадение не подтверждается. Одно название не доказывает идентичность; отличия варианта и комплектации обозначаются явно. Подтверждение одинакового варианта за пределами собранных данных не требуется.

Результат содержит площадку, название, URL, совпавшие признаки, отличия, цену, её условие и дату. Экономии рассчитывать только при одинаковых регионе, валюте и условиях. Недостаточные данные обозначать явно.

5 Пользовательские SQL-запросы

Реализовать минимум десять именованных запросов отдельными файлами. Новый запрос добавляется SQL-файлом и записью его параметров в реестре; изменение парсеров не требуется. Произвольный SQL из пользовательской строки не формировать.

Код	Запрос	Параметры
Q01	Аналоги на других площадках	ID товара, ANC, микрофон
Q02	Фильтрация по характеристикам и цене	подключение, форм-фактор, диапазон цены
Q03	Одинаковая модель на разных площадках	бренд, модель
Q04	Минимальная текущая цена по площадкам	признаки, регион, тип цены
Q05	Пять самых дешёвых подходящих товаров	фильтры, свежесть
Q06	История цены товара	ID, контекст, период
Q07	Снижение между двумя последними ценами	процент снижения, контекст
Q08	Рейтинг и количество отзывов	минимальные значения
Q09	Средняя, минимальная и максимальная цена	признаки, контекст, свежесть

Код	Запрос	Параметры
Q10	Устаревшие данные и результаты сборов	возраст, площадка, статус

Запросы должны демонстрировать JOIN, WHERE, GROUP BY, агрегатные и оконные функции. Для Q07 использовать LAG; для актуальных цен — ROW_NUMBER либо DISTINCT ON. Историю разных условий цены не смешивать.

Фильтры комбинируются. Пример: беспроводные полноразмерные наушники с микрофоном до 5000 рублей на двух выбранных площадках с данными не старше суток. Неизвестные характеристики и наличие не подменять подтверждёнными значениями.

Параметры проверять по типу и диапазону. Для прямых запросов через psql предусмотреть отдельную роль только с правами SELECT.

6 Сбор и автоматизация

Достаточно одной категории и минимум 30 карточек каждой площадки. Адаптеры возвращают единый формат. Получение страницы отделяется от разбора; HTML для диагностики не должен содержать cookies и данные личного кабинета.

Обязательны повторный запуск, сохранение истории, пропуск свежих данных и журнал ошибок. Обычный сбор выполняется одной командой без ручного копирования карточек. Расписание допускается через стандартный планировщик ОС; собственный сервис задач не требуется.

При 403 сбор площадки останавливается с понятной причиной. Импорт файла не считается успешным живым сбором. Перед разработкой адаптера проверить карточку и выдачу; при недоступности согласовать замену площадки. Импорт позволяет независимо завершить БД и запросы.

7 Технические проверки

1. Схема создаётся на чистой PostgreSQL, программа запускается по README.
2. Загружено минимум 90 карточек, по 30 с каждой из трёх согласованных площадок.
3. Повторный сбор не дублирует карточки и сохраняет новые наблюдения.
4. Q01–Q10 дают ожидаемые результаты на контрольных данных.
5. Для 10 контрольных товаров возвращаются известные аналоги и исключаются все кандидаты с противоречащими обязательными признаками.
6. Проверены NULL, условия цены, неизвестное наличие, отказ площадки и откат транзакции.
7. Текущие цены и история выбираются корректно; Q07 не смешивает контексты.
8. CSV содержит заголовки и кириллицу; секретов в данных и журнале нет.
9. Предоставлены пример pg_dump и инструкция восстановления.

Контрольные данные включают подходящие и неподходящие аналоги, несколько цен одного товара и неизвестные поля. SQL проверяется на PostgreSQL. Достаточно unittest либо pytest; нагрузочные испытания и сложная инфраструктура CI не требуются.

8 Запуск и восстановление

README должен содержать команды установки зависимостей, создания схемы PostgreSQL, импорта данных, сбора, поиска и экспорта. Схема и пользовательские запросы хранятся в SQL-файлах. Для проверки подключения предусмотреть отдельную команду.

Резервное копирование выполняется через pg_dump. В инструкции восстановления указать создание БД и загрузку копии. Примеры данных и конфигурации не должны содержать паролей и cookies.

Подписи

Студент _____ Дата _____ Подпись _____

Студент _____ Дата _____ Подпись _____

Преподаватель _____ Дата _____ Подпись _____